

Backend(Sunday 01/03/2026) :

Exceptions.py : this file is used for error occurs in our API and it creates a centralized error handling system for our Django REST framework API, and it catches all exceptions and returns consistent well formatted error responses and instead of using Django's default error pages we will get a standardized JSON response like this one here :

```
{
    "error": true,
    "status_code": 400,
    "message": "Une erreur est survenue",
    "details": {
        "email": ["This field is required."],
        "password": ["This field is required."]
    }
}
```

And it uses two main paths for this :

Exception Occurs

↓

custom_exception_handler(exc, context)

↓

Call default DRF exception handler

↓

| — DRF HANDLED → Returns response

| ↓

| Customize the response format

| ↓

| Log the error

| ↓

| Return formatted response

|

└─ UNHANDLED → Returns None

↓

Create 500 error response

↓

Log full exception

↓

Return error (with details only in DEBUG)

And in my code we can see that :

```
import logging
from django.conf import settings
from rest_framework import status
from rest_framework.response import Response
from rest_framework.views import exception_handler
```

```
logger = logging.getLogger(__name__)
```

So we set up logging to track errors and in our code we have DRF handled errors 400,401,402,403,404..... and unhandled errors 500 and that's by this :

```
response = exception_handler(exc, context)

if response is not None:
    # Personnaliser la réponse d'erreur
    custom_response_data = {
        "error": True,
        "status_code": response.status_code,
        "message": "Une erreur est survenue",
        "details": response.data,
    }

    # Logger l'erreur
    logger.error(
        f"API Error: {exc.__class__.__name__} - {str(exc)}",
        extra={
            "status_code": response.status_code,
            "path": context["request"].path,
            "method": context["request"].method,
        },
    )
```

```
response.data = custom_response_data
```

This one here is for the handled errors that doesn't have any relation with our server

```
else:
    # Erreur non gérée par DRF
    logger.exception(f"Unhandled exception: {exc}")

    response = Response(
        {
            "error": True,
            "status_code": status.HTTP_500_INTERNAL_SERVER_ERROR,
            "message": "Erreur serveur interne",
            "details": str(exc) if settings.DEBUG else "Une erreur est
survenue",
        },
        status=status.HTTP_500_INTERNAL_SERVER_ERROR,
    )

    return response
```

And this one here is for unhandled errors like 500 internal server error

HTTP Status	When It Happens	Example
400 Bad Request	Validation error	Missing required field
401 Unauthorized	Not authenticated	No token provided
403 Forbidden	No permission	Regular user accessing admin
404 Not Found	Resource missing	Wrong URL or ID
500 Internal Error	Server crash	Database down, bug in code

As a summary we can say that the benefits of our file here are :

Benefit	Description
Consistency	Every error has the same JSON structure
Logging	All errors are automatically logged

Security	Hides internal details in production
Debugging	Full details in development
UX Friendly	Clear error messages for frontend
Centralized	One place to modify all error handling

Filter.py: this code creates a filter system for the trajet model that allows users to search and filter trips based on specific criteria and we can consider it as a search function for our web site .

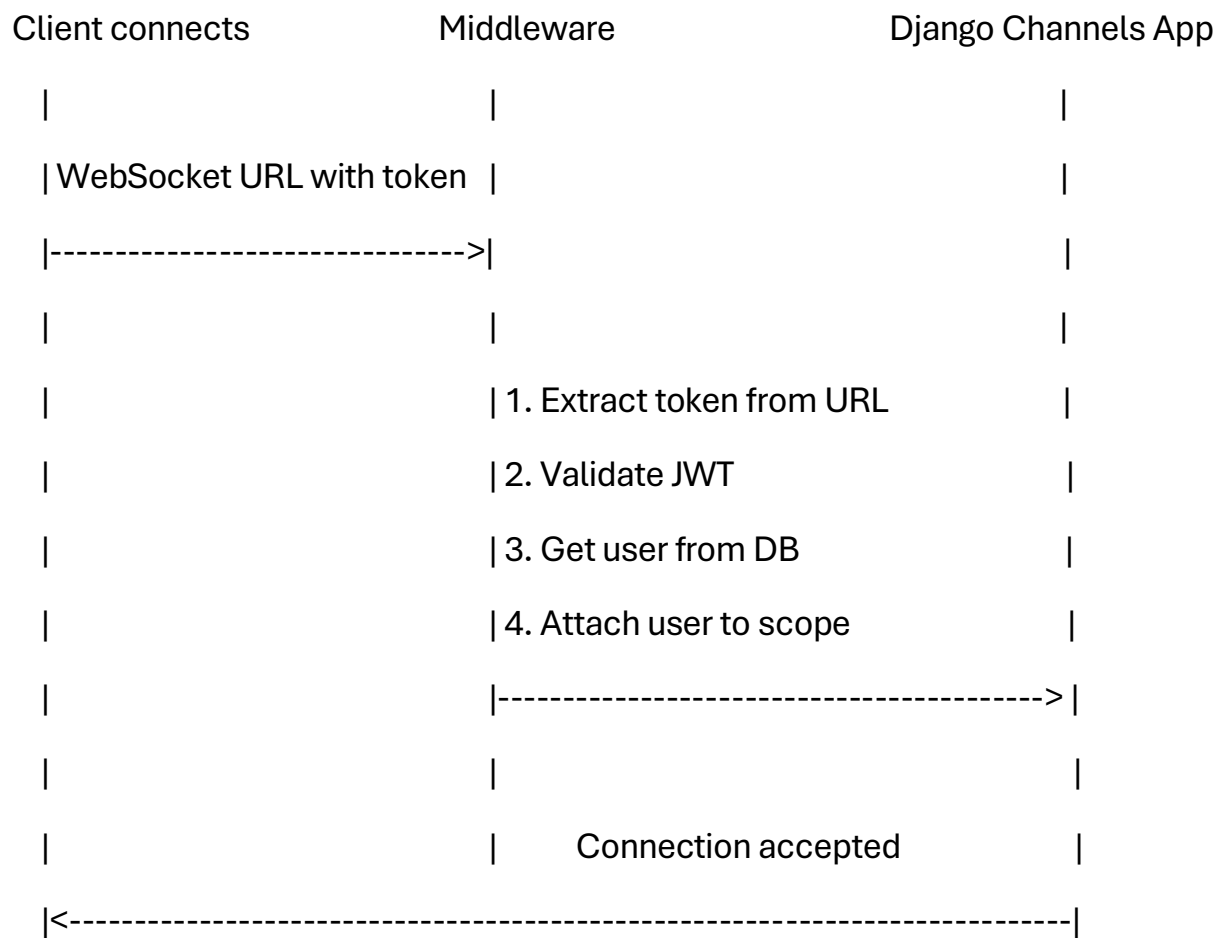
Field	Filters by	Example
ville_depart	Departure city	?ville_depart=Alger
ville_arrivee	Arrival city	?ville_arrivee=Oran
date	Trip date	?date=2024-03-15
conducteur	Driver (ID)	?conducteur=42
status	Trip status	?status=PLANNED

And for the API :

GET /api/trajets/?ville_depart=Alger&ville_arrivee=Oran&date=2024-03-15

And by this filter a user can search and filter all trips in our site web by departure city and arrival city and date and driver and status and all of this through a simple URL parameters.

middleware_websocket.py: this middleware authenticates websocket connections using JWT tokens passed in the URL allowing real time features like chat or notifications for authenticated users and know who is sending messages in this case



Security features :

Feature	Purpose
Token validation	Ensures token is valid and not tampered
User lookup	Verifies user still exists in database
Anonymous fallback	Allows public WebSocket endpoints
Error handling	Never crashes, always returns valid user

1_This middleware **automatically authenticates WebSocket connections** by extracting JWT tokens from the URL, validating them, and attaching the user to the connection scope so all your WebSocket consumers can easily know who's connected.

Middleware.py :in this file we can find four middleware components intercept every HTTP request to add logging, track activity, handle cors, and implement rate limiting

1_requestloggingmiddleware – logs all API requests :

```
class RequestLoggingMiddleware:
    """
    Middleware pour logger toutes les requêtes API
    """

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        # Log avant la requête
        start_time = timezone.now()

        # Traiter la requête
        response = self.get_response(request)

        # Log après la requête
        duration = (timezone.now() - start_time).total_seconds()

        log_data = {
            "method": request.method,
            "path": request.path,
            "status": response.status_code,
            "duration": f"{duration:.3f}s",
            "user": getattr(request.user, "email", "Anonymous"),
            "ip": self.get_client_ip(request),
        }

        if response.status_code >= 400:
            logger.warning(f"Request failed: {log_data}")
        else:
            logger.info(f"Request processed: {log_data}")

        return response

    @staticmethod
    def get_client_ip(request):
        """Récupère l'IP réelle du client"""
        x_forwarded_for = request.META.get("HTTP_X_FORWARDED_FOR")
        if x_forwarded_for:
```

```

        ip = x_forwarded_for.split(",")[0]
    else:
        ip = request.META.get("REMOTE_ADDR")
    return ip

```

2_useractivitymiddleware – tracks user activity :

```

class UserActivityMiddleware:
    """
    Middleware pour tracker la dernière activité des utilisateurs
    """

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)

        # Mettre à jour last_activity pour les utilisateurs authentifiés
        if request.user.is_authenticated:
            # Utiliser update() pour éviter de déclencher les signaux
            User.objects.filter(pk=request.user.pk).update(updated_at=timezone.now())

        return response


class CORSMiddleware:
    """
    Middleware personnalisé pour gérer CORS de manière plus fine
    """

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        response = self.get_response(request)

        # Ajouter des headers CORS personnalisés si nécessaire
        if request.path.startswith("/api/"):
            origin = request.META.get("HTTP_ORIGIN", "")

            # Liste des origines autorisées
            allowed_origins = [
                "http://localhost:3000",

```

```

        "http://127.0.0.1:3000",
        "https://dzcarpool.com",
    ]

    if origin in allowed_origins:
        response["Access-Control-Allow-Origin"] = origin
        response["Access-Control-Allow-Credentials"] = "true"

    return response

```

3_corsmiddleware – manages cross-origin requests

```

allowed_origins = [
    "http://localhost:3000",      # React dev server
    "http://127.0.0.1:3000",      # Local dev alternative
    "https://dzcarpool.com",      # Production frontend
]

```

4_ratelimitmiddleware – prevents abuse

```

# Simple rate limiting: max 100 requests per minute per IP
if cache_entry["count"] > 100:
    return JsonResponse({"error": "Rate limit exceeded"}, status=429)

```

All those request flow can be handled together through all middlewares :

Request Incoming

↓

1. RequestLoggingMiddleware (start timer)

↓

2. UserActivityMiddleware (check if authenticated)

↓

3. CORSMiddleware (add CORS headers if needed)

↓

4. RateLimitMiddleware (check request count)

↓

View/Controller

↓

5. RequestLoggingMiddleware (log duration and result)

↓

Response Outgoing

So each middleware has its own use and its own purpose just like anything in my backend and i can summary it i this table :

Middleware	Purpose	Real-World Use
RequestLogging	Monitoring & debugging	Track API usage, find errors
UserActivity	User analytics	Show "online" status, user engagement
CORS	Security	Allow only your frontend to call API
RateLimit	Protection	Stop bots and brute force attacks

RequestLoggingMiddleware: Logs every API request with details (who, what, when, how long) so you can monitor and debug your application.

UserActivityMiddleware : Updates the "last seen" timestamp for logged-in users whenever they make a request.

CORSMiddleware : Controls which websites can access your API (only allowing your own frontend domains).

RateLimitMiddleware : Prevents abuse by limiting how many requests each IP address can make (100 per minute).

These middlewares automatically log all requests, track user activity, control which websites can access your API, and prevent abuse through rate limiting .

SAIDI_MAHMOUD_2CP7